LINKED LIST

USING PYTHON

python

SIMPLE EXPLANATION WITH CODE

EASY TO UNDERSTAND | PYTHON CODE | INTERVIEW FRIENDLY

SUBSCRIBE NOW!

What is a Linked List?

- A linear data structure
- Elements are stored in **nodes**
- Each node contains:
  - **Data**
  - **Pointer (reference)** to next node
- 👉 Unlike arrays, elements are **not stored in contiguous memory**

Type of Linked llst

- Singly
- Doubly
- Circuler

Structure of a Node in (Singly Linked list)

- Each node has:
  - data → value
  - next → address of next node

NODE

Create a Node Class

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

Create Linked List Class

```
class LinkedList:
    def __init__(self):
        self.head = None
```

Insert at Beginning

```
def insert_at_beginning(self, data):
    new_node = Node(data)
    new_node.next = self.head
    self.head = new_node
```

Insert at End

```
def insert_at_end(self, data):
    new_node = Node(data)

    if self.head is None:
        self.head = new_node
        return

    temp = self.head
    while temp.next:
        temp = temp.next

    temp.next = new_node
```

Display Linked List

```
def display(self):
    temp = self.head
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

Complete Example

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_beginning(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return

        temp = self.head
        while temp.next:
            temp = temp.next
        temp.next = new_node

    def display(self):
        temp = self.head
        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next
        print("None")
```

```
# Usage
ll = LinkedList()
ll.insert_at_beginning(10)
ll.insert_at_beginning(5)
ll.insert_at_end(20)
ll.insert_at_end(30)

ll.display()
```

SINGLY LINKED LIST & COMMON OPERATIONS

1. SINGLY LINKED LIST (SLL)

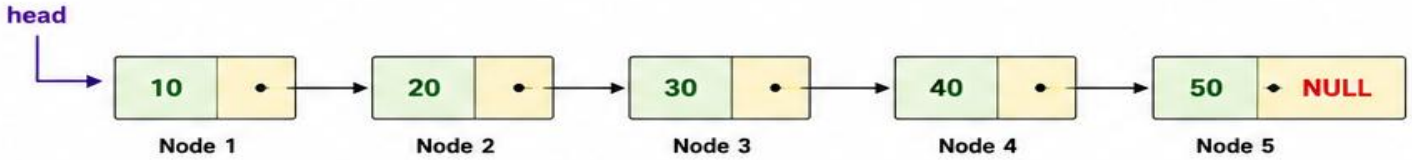
A linear data structure where each node contains:

- Data (Value)
- Address (Link) of next node

NODE STRUCTURE

HEAD
Head is a pointer that stores the address of the first node.

EXAMPLE OF SINGLY LINKED LIST



TRAVERSAL (Display)

Start from head and visit each node by following Next pointer until NULL.

Output: 10 → 20 → 30 → 40 → 50

COMPLEXITY

- Traversal : O(n)
- Search : O(n)
- Insertion : O(1) *
- Deletion : O(1) *

* At beginning. In middle/end requires O(n) to find position.

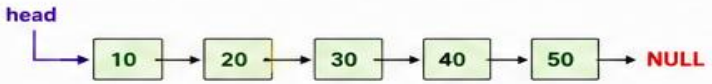
ADVANTAGES

- Dynamic size
- Efficient insertions/deletions
- Memory is allocated as needed

2. COMMON OPERATIONS

2.1 TRAVERSAL (DISPLAY)

Visit each node from head to NULL.

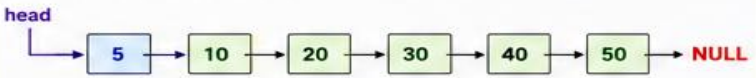


Output: 10 → 20 → 30 → 40 → 50

2.2 INSERT AT BEGINNING

New node (5) is inserted before the first node.

Steps:
1. newNode->Next = head
2. head = newNode



2.3 INSERT AT END

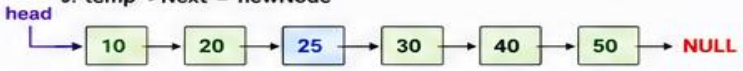
New node (60) is inserted at the last.

Steps:
1. Traverse to last node
2. last->Next = newNode
   newNode->Next = NULL

2.4 INSERT AT POSITION (Pos = 3, New Node = 25)

Insert 25 at position 3 (between 20 and 30).

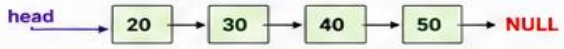
Steps:
1. Traverse to (pos-1)th node (here node with 20)
2. newNode->Next = temp->Next
3. temp->Next = newNode



2.5 DELETE FROM BEGINNING

Delete the first node (10).

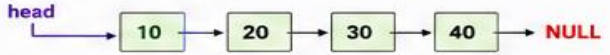
Steps:
1. temp = head
2. head = head->Next
3. free(temp)



2.6 DELETE FROM END

Delete the last node (50).

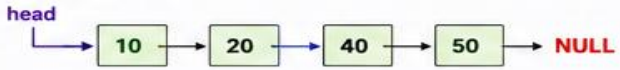
Steps:
1. Traverse to second last node (40)
2. temp->Next = NULL
3. free(lastNode)



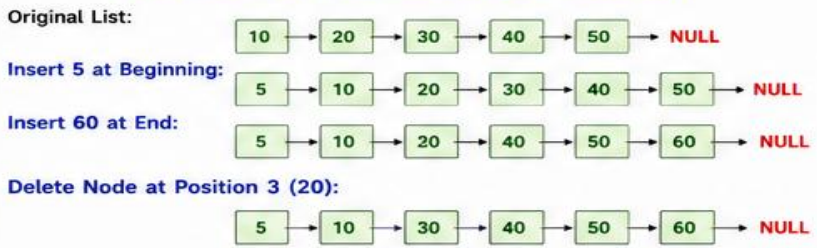
2.7 DELETE AT POSITION (Pos = 3)

Delete node at position 3 (node with 30).

Steps:
1. Traverse to (pos-1)th node (20)
2. temp = target node (30)
3. prev->Next = temp->Next
4. free(temp)



FULL EXAMPLE (AFTER MULTIPLE OPERATIONS)



IMPORTANT NOTES

- In Singly Linked List, each node points to the next node only.
- Last node's Next pointer always contains NULL.
- Always start from head to access any node.
- Insertions/Deletions are efficient when the position is known.

LEGEND

DOUBLY LINKED LIST (DLL) & COMMON OPERATIONS

1. DOUBLY LINKED LIST (DLL)

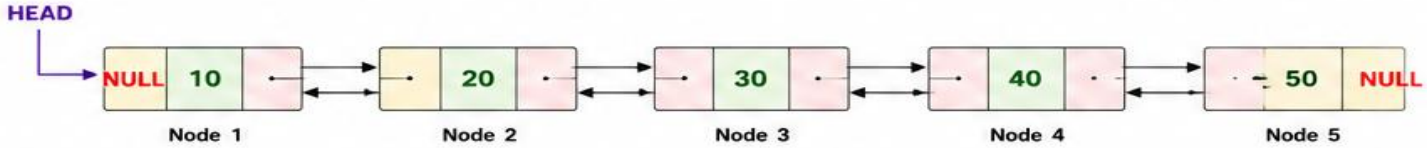
A linear data structure where each node contains:

- Data (Value)
- Address (Link) of next node
- Address (Link) of previous node

NODE STRUCTURE

HEAD
Pointer to the first node.
In DLL, we can traverse in both forward and backward directions.

EXAMPLE OF DOUBLY LINKED LIST



TRAVERSAL

Forward (from Head to NULL): 10 ↔ 20 → 30 → 40 → 50

Backward (from Last to NULL): 50 ↔ 40 ↔ 30 ↔ 20 ↔ 10

(We can traverse in both directions)

COMPLEXITY

- Traversal : O(n)
- Search : O(n)
- Insertion : O(1) *
- Deletion : O(1) *

* At beginning, middle or end (when position is known)

ADVANTAGES

- Efficient insertion and deletion at both ends and at any position (when position is known)
- Bidirectional traversal
- More flexible than SLL

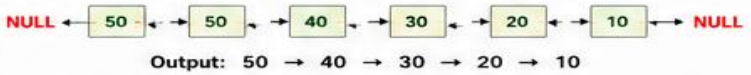
2. COMMON OPERATIONS

2.1 TRAVERSAL (DISPLAY)

Forward Traversal (Head to End)

Output: 10 → 20 → 30 → 40 → 50

Backward Traversal (End to Head)



Output: 50 → 40 → 30 → 20 → 10

2.2 INSERT AT BEGINNING

Insert new node (5) before the first node.

Steps:

1. newNode->Next = head
2. newNode->Prev = NULL
3. head->Prev = newNode
4. head = newNode

2.3 INSERT AT END

Insert new node (60) at the last.

Steps:

1. Traverse to last node
2. last->Next = newNode
3. newNode->Prev = last
4. newNode->Next = NULL

2.4 INSERT AT POSITION (Pos = 3, New Node = 25)

Insert 25 at position 3 (between 20 and 30).

Steps:

1. Traverse to (pos-1)th node (20)
2. newNode->Next = current->Next (30)
3. newNode->Prev = current (20)
4. (current->Next)->Prev = newNode
5. current->Next = newNode

2.5 DELETE FROM BEGINNING

Delete the first node (10).

Steps:

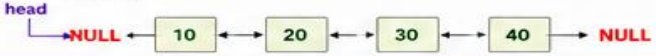
1. temp = head
2. head = head->Next
3. head->Prev = NULL
4. free(temp)

2.6 DELETE FROM END

Delete the last node (50).

Steps:

1. Traverse to second last node (40)
2. temp = 40->Next (50)
3. 40->Next = NULL
4. free(temp)



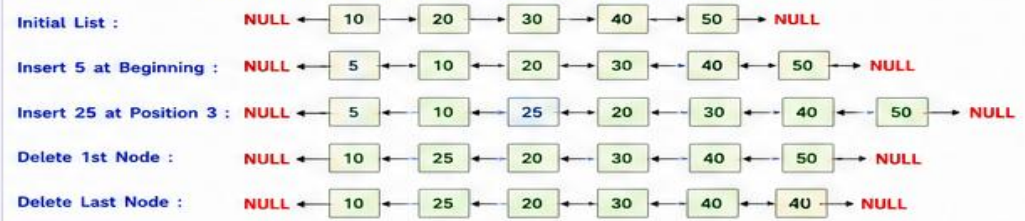
2.7 DELETE AT POSITION (Pos = 3)

Delete node at position 3 (node with 30).

Steps:

1. Traverse to node at position 3 (30)
2. (30->Prev)->Next = 30->Next (40)
3. (30->Next)->Prev = 30->Prev (20)
4. free(30)

FULL EXAMPLE (AFTER MULTIPLE OPERATIONS)



IMPORTANT NOTES

- Each node has two links: Prev and Next.
- Prev of first node is NULL.
- Next of last node is NULL.
- Traversal possible in both forward and backward directions.
- DLL requires extra memory for Prev link.

LEGEND

CIRCULAR LINKED LIST (CLL) & COMMON OPERATIONS

1. CIRCULAR LINKED LIST (CLL)

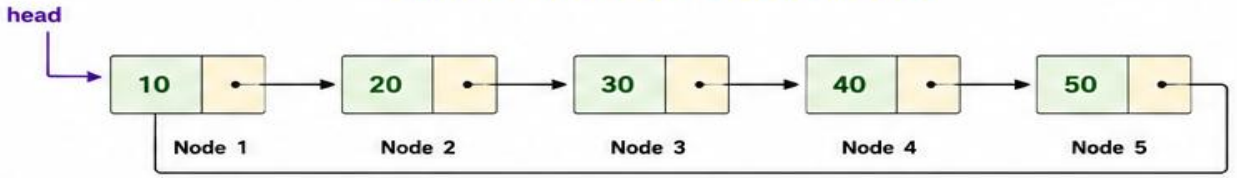
A linear data structure in which the last node points back to the first node, forming a circle.

- Each node contains:
  - Data (Value)
  - Address (Link) of next node

NODE STRUCTURE

HEAD
Head is a pointer that stores the address of the first node.

EXAMPLE OF CIRCULAR LINKED LIST



TRAVERSAL (Display)

Start from head and visit each node by following Next pointer until we reach the head node again.

Output: 10 → 20 → 30 → 40 → 50 (back to 10)

COMPLEXITY

Traversal	O(n)
Search	O(n)
Insertion	O(1) *
Deletion	O(1) *

* At beginning or at last (when position is known)

ADVANTAGES

- Last node is connected to first node.
- Efficient insertion and deletion at beginning or end.
- Useful for circular applications (e.g., round-robin scheduling).

2. COMMON OPERATIONS

2.1 TRAVERSAL (DISPLAY)

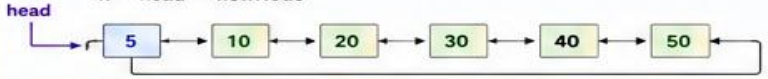
Visit each node from head and stop when we come back to head.

Output: 10 → 20 → 30 → 40 → 50

2.2 INSERT AT BEGINNING

Insert new node (5) before the first node.

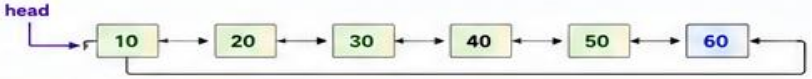
Steps:
1. newNode->Next = head
2. Traverse to last node (50)
3. last->Next = newNode
4. head = newNode



2.3 INSERT AT END

Insert new node (60) after the last node.

Steps:
1. Traverse to last node (50)
2. last->Next = newNode (60)
3. newNode->Next = head



2.4 INSERT AT POSITION (Pos = 3, New Node = 25)

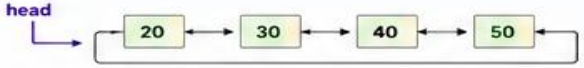
Insert 25 at position 3 (between 20 and 30).

Steps:
1. Traverse to (pos-1)th node (20)
2. newNode->Next = current->Next (30)
3. current->Next = newNode

2.5 DELETE FROM BEGINNING

Delete the first node (10).

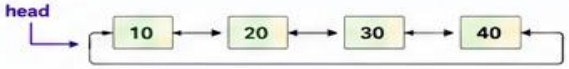
Steps:
1. Traverse to last node (50)
2. head = head->Next (20)
3. last->Next = head



2.6 DELETE FROM END

Delete the last node (50).

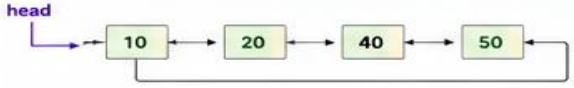
Steps:
1. Traverse to second last node (40)
2. secondLast->Next = head (10)
3. free(lastNode)



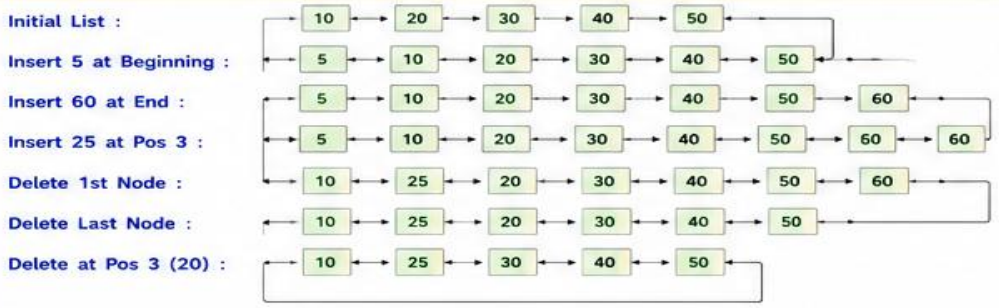
2.7 DELETE AT POSITION (Pos = 3)

Delete node at position 3 (node with 30).

Steps:
1. Traverse to (pos-1)th node (20)
2. temp = (pos)th node (30)
3. current->Next = temp->Next (40)
4. free(temp)



FULL EXAMPLE (AFTER MULTIPLE OPERATIONS)



IMPORTANT NOTES

- In CLL, last node's next always points to head.
- Traversal stops when we reach head again.
- Only one pointer (next) is used in each node.
- Best for applications where we need circular access.

LEGEND